

eSC-V: A Bare-Metal RISC-V Microcontroller With Hardware-Enforced Control-Flow Integrity

Arjun Salunkhe^{1*}, Atharva Bhosale¹, Sourabh Bansode¹, Srushti Shinde¹ and Kanchan Vaidya¹

¹ Ajeenkya DY Patil SOE, Pune, MH, India

***Corresponding Author:** Arjun Salunkhe, arjun.salunkhe@proton.me

Abstract. The rapid expansion of IoT and robotics has shifted critical control tasks to bare-metal microcontrollers, yet these Microcontroller Class (M-mode) devices remain highly vulnerable to Code-Reuse Attacks (CRAs) such as ROP and JOP. While high-performance systems utilize complex OS-based defenses, resource-constrained cores often lack hardware-enforced Control-Flow Integrity (CFI).

This study introduces eSC-V, a 32-bit RISC-V (*RV32I Zicsr Zicfilp Smcfiss Smpmpnd*) CPU designed from scratch in VHDL to bridge this security gap. The primary objective is to evaluate the feasibility of integrating the ratified Zicfilp extension and the machine-mode Draft Smcfiss extension, which utilizes Smpmpnd for indirect CSR access to isolate the shadow stack memory. The core features a five-stage pipeline, has been developed using an end-to-end open-source toolchain and synthesized on a Sipeed Tang Primer 20K FPGA at 60 MHz. The architecture was architecturally verified using the RISCOF framework against the Sail and Spike formal models. Findings demonstrate that robust, standards-compliant forward and backward-edge protection is achievable in minimalist M-mode processors with negligible logic overhead. eSC-V establishes a practical, auditable blueprint for securing next-generation bare-metal RISC-V platforms.

Keywords: *RISC-V, Control-Flow Integrity (CFI), Smcfiss, Zicfilp, Bare-Metal Security, Five-Stage Pipeline, Open Source*

Introduction:

The embedded systems landscape is undergoing a fundamental transformation, driven by the convergence of the Internet-of-Things (IoT) and advanced robotics. Devices that were once isolated are now interconnected nodes within a global ecosystem, supporting high-stakes applications ranging from autonomous mobile robots (AMRs) and industrial collaborative bots to smart grids and medical telemetry. While this connectivity has expanded functionality and enabled real-time coordination, it has also significantly increased the attack surface of microcontroller-based systems. Unfortunately, the architectural security mechanisms protecting these Machine-mode (M-mode) devices have not evolved at the same pace as the emerging threat landscape, leaving them vulnerable to sophisticated exploits [2], [3].

A. The vulnerability of bare-metal

Since most embedded firmware uses C or C++—languages prioritizing efficiency over memory safety—devices remain vulnerable to memory corruption like buffer overflows. In stack-based overflows, writing beyond a local buffer can overwrite adjacent memory, including the return address used to resume execution after a function call.

If an attacker overwrites this address, the subsequent return instruction transfers control to an arbitrary location. This enables Return-Oriented Programming (ROP), where adversaries chain existing instruction sequences to construct malicious paths without injecting new code.

Similarly, Jump-Oriented Programming (JOP) targets forward-edge control flow by corrupting indirect targets like function pointers. Recent studies demonstrate JOP's feasibility on RISC-V platforms, confirming that open ISA architectures are not inherently immune to control-flow hijacking [4].

B. Limitations of software-based defenses in microcontrollers

High-performance systems mitigate threats using OS-level defenses like Address Space Layout Randomization (ASLR), virtual memory, and shadow stacks. Bare-metal microcontrollers, however, lack Memory Management Units (MMUs) and operate in a flat physical address space, precluding these protections.

Software-based Control-Flow Integrity (CFI) uses compiler-instrumented runtime checks for indirect branches and returns. While effective, the resulting instruction overhead introduces timing jitter and latency, violating deterministic execution requirements in real-time workloads.

Furthermore, bare-metal RISC-V systems operating entirely in Machine Mode (M-Mode) lack privilege separation. Consequently, a single control-flow hijack grants an attacker unrestricted access to physical memory, peripherals, cryptography, and configuration registers, severely amplifying the impact of such attacks in microcontroller environments.

C. The eSC-V approach

To address this security deficit, we introduce eSC-V, a secure microcontroller-class RISC-V SoC that embeds Control-Flow Integrity (CFI) directly into its pipeline. Avoiding external coprocessors or heavy software instrumentation, eSC-V integrates the ratified Zicfilp and draft Smcfiss extensions into a classical five-stage in-order RV32I architecture.

Zicfilp enforces forward-edge integrity by requiring indirect transfers to target defined landing pad instructions. Smcfiss ensures backward-edge integrity via a hardware-protected shadow stack, using the Smpmpind extension's indirect CSR access for PMP-based memory isolation.

Through this design, eSC-V establishes a practical, reproducible reference architecture for secure edge devices operating in resource-constrained, bare-metal environments.

Methodology:

The following section details the architectural design and implementation of the eSC-V core, a custom 32-bit RV32I Zicsr processor built from scratch in VHDL. The methodology focuses on the microarchitectural integration of hardware-enforced security primitives into a standard five-stage pipeline and the subsequent SoC deployment.

A. Theoretical Framework: RISC-V CFI Extensions

In June 2024, RISC-V ratified the Zicfilp extension [5], [6], [16] to standardize forward-edge Control-Flow Integrity. Concurrently, the draft Smcfiss and Smpmpind extensions extend backward-edge protection (Zicfiss) to Machine-mode environments. Together, these open-standard security milestones form the theoretical foundation of eSC-V.

Zicfilp: Landing pads for forward-edge protection. The Zicfilp extension constrains the targets of indirect control transfers (e.g., *jalr*). When an indirect jump is executed, the processor enters a Landing-Pad-Enabled (LPE) tracking state. In this state, the subsequent instruction must be a valid *lpad*. If any other instruction is encountered, a software-check exception is raised.

The *lpad* instruction embeds a 20-bit label. The expected label, prepared by the calling context, is compared against the label encoded in the target *lpad*. This enables fine-grained forward-edge validation, restricting indirect calls to valid entry points and significantly reducing the freedom available to an attacker attempting Jump-Oriented Programming (JOP) [4]. In eSC-V, activation of this mechanism is controlled through the *mseccfg* CSR.

Smcfiss and Smpmpind: Shadow stack for backward-edge protection. A shadow stack is a hardware-protected memory region operating parallel to the standard data stack, exclusively storing function return addresses. Addresses are pushed to both stacks during a call and validated against each other upon return, neutralizing Return-Oriented Programming (ROP) by immediately detecting primary stack corruption.

The RISC-V Zicfiss extension standardizes this using instructions like *sspsh* and *sspopchk*. However, baseline Zicfiss targets application processors, relying on MMUs for isolation. For bare-metal microcontrollers, the draft Smcfiss extension enables Machine-mode backward-edge protection instead. It introduces the MSSE field in the *mseccfg* CSR to activate the shadow stack [18], tracked by a dedicated Shadow Stack Pointer (*ssp*) CSR.

Lacking virtual memory, Smcfiss isolates the shadow stack using Physical Memory Protection via the Smpmpind extension [18]. Smpmpind introduces an extended-attributes (*E*) field in the *pmpcfg* register, configured via indirect CSR access. When *mseccfg.MSSE=1*, a PMP entry with *E=1* and *XWR=010* designates an M-mode shadow stack region.

To guarantee backward-edge integrity, hardware enforces a rigorous set of access rules that effectively transform the shadow stack into a protected environment. It ensures immutability by triggering access faults for any standard stores or implicit accesses, such as instruction fetches, that attempt to interfere with the stack. Furthermore, the system maintains exclusive access by restricting shadow stack instructions strictly to valid M-mode shadow regions. This is

bolstered by strict read permissibility, which limits the region's interaction to either explicit reads or specialized shadow stack instructions, preventing unauthorized tampering while allowing for necessary verification.

B. eSC-V Microarchitectural Integration

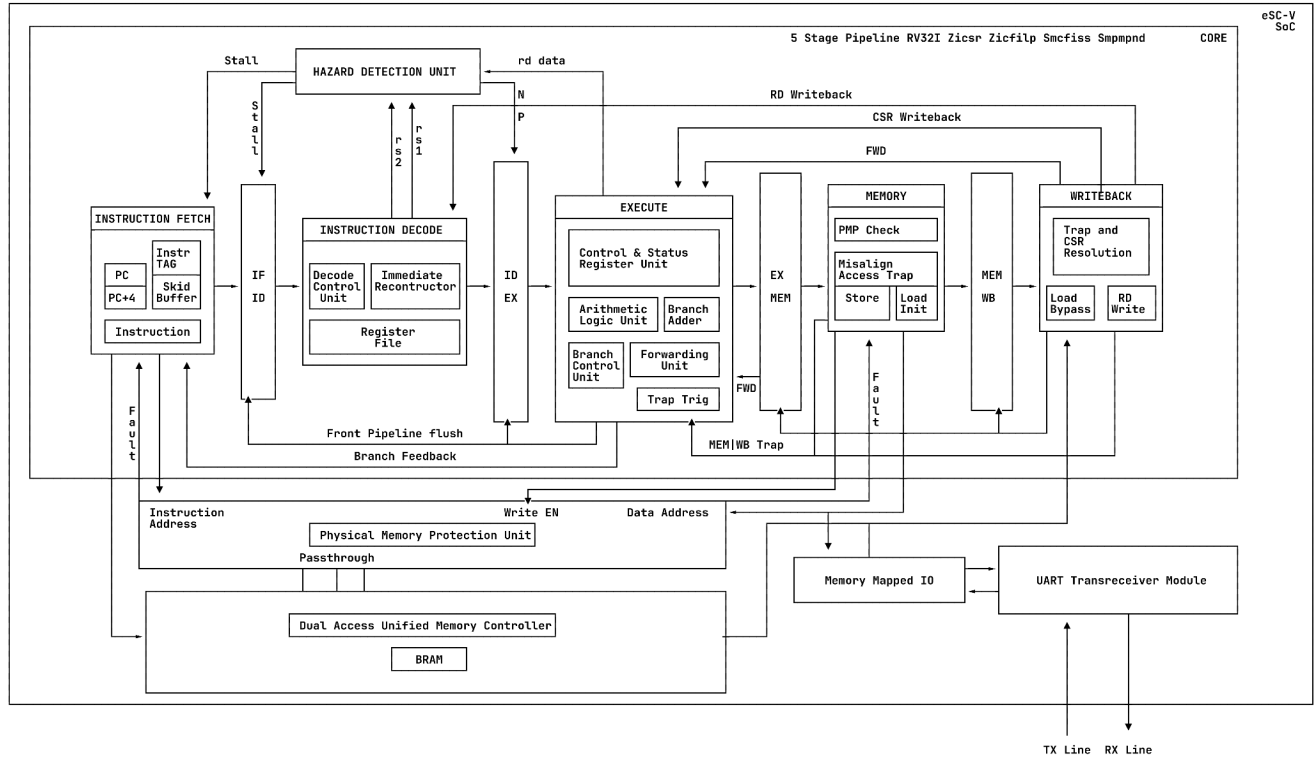


Fig. 1. Baseline five-stage RV32I pipeline organization (IF–ID–EX–MEM–WB) used in eSC-V.

The eSC-V core implements a five-stage in-order pipeline (IF-ID-EX-MEM-WB) for the RV32I Zicsr instruction set. Integrating hardware-enforced CFI into this pipeline required custom exception handling, a unified memory interface, and targeted hazard mitigation strategies.

Figure 1 illustrates the baseline five-stage RV32I pipeline organization used in the eSC-V architecture. Rather than detailing the standard RISC-V baseline, this section highlights the specific microarchitectural modifications introduced to support hardware-enforced CFI, along with the broader SoC integration.

Five-stage pipeline architecture. The eSC-V core is built upon a custom, VHDL-designed five-stage in-order pipeline executing the RV32I and Zicsr instruction sets. The pipeline follows the classic IF-ID-EX-MEM-WB organization with strict modular stage separation.

The classic five-stage pipeline architecture streamlines instruction processing through a sequence of specialized functional units. It initiates with Instruction Fetch (IF), which retrieves instructions from a unified dual-port block memory controller while simultaneously handling program counter increments. This is followed by Instruction Decode (ID), where the system decodes the fetched instruction, reconstructs immediates, reads source operands from the register

file, and generates the control signals required for downstream execution. In the Execute (EX) stage, an Arithmetic Logic Unit (ALU) and a dedicated branch adder compute results and resolve control-flow targets, supported by a Control and Status Register (CSR) unit for rapid evaluation. The process continues with the Memory (MEM) stage, which interfaces with the data port of the memory controller to perform load and store operations. Finally, the Writeback (WB) stage commits the final ALU results, bypassed memory load data, and CSR state updates back to the architectural register file, completing the instruction cycle.

To maintain single-cycle throughput without unnecessary stalls, the architecture integrates a centralized Hazard Detection Unit to manage load-use data dependencies via pipeline stalls. Simultaneously, a comprehensive Forwarding Unit spans the EX and MEM stages, resolving General Purpose Register (GPR) and CSR data hazards dynamically.

Zicfilp enforcement and two-stage fault handling. To minimize critical path delays while ensuring security, eSC-V implements a distributed Zicfilp tracking and a custom "two-stage flush" fault architecture.

The control-flow integrity (CFI) mechanism in this architecture follows a rigorous three-phase process to secure indirect jumps. It begins with Tracking Initiation, where the Execution (EX) stage identifies indirect jumps (such as *JALR*) and asserts an *elp* (Expected Landing Pad) signal to the Instruction Fetch (IF) stage. This moves into Tagging & Verification, during which the IF stage tags the target instruction; if the EX stage later determines this instruction is not a valid *lpad*, it assigns an *LPAD_FAULT* and squashes the instruction's control signals. To protect the architectural state during these faults, a two-Stage Pipeline Flush is employed. If a fault is detected early (in the IF, ID, or EX stages), the front-end is immediately flushed to block unauthorized memory or CSR accesses. The squashed faulting instruction then travels to the Writeback (WB) stage, where its retirement triggers a final, full pipeline flush and redirects the system to the *mtvec* trap handler for exception processing.

Smcfiss and passthrough PMP integration. The Machine-mode shadow stack is enforced through a tight coupling between the memory pipeline and the Physical Memory Protection (PMP) unit.

To round out the memory and validation logic, the architecture implements a dual-layered security check between the memory interface and the final commit. The Passthrough PMP (Physical Memory Protection) acts as an inline four-entry filter between the Memory (MEM) stage and the unified block memory controller, continuously evaluating access privileges in real-time. If an unauthorized store attempts to write to the designated shadow stack region, the PMP unit immediately blocks the data path and squashes the MEM signals to trigger an access fault. Complementing this, Writeback Stage Validation handles the timing constraints of the memory controller's one-cycle load delay; because load data is bypassed in the WB stage, the critical backward-edge validation for *sspopchk* is performed here. This ensures that the loaded shadow stack return address is verified against the architectural link register in the WB stage before the instruction is finally permitted to retire.

The *lpad* instruction's 20-bit label is reconstructed during decoding and propagated forward for verification. Shadow stack instructions generate dedicated control signals that regulate privilege checks and memory access constraints.

Hazard mitigation and CSR management. The introduction of the *ssp* (Shadow Stack Pointer) and *mseccfg* registers requires careful data hazard management to avoid pipeline stalls. While the CSR unit resides in the EX stage for swift execution, CSR state changes are strictly committed only when the instruction successfully retires in the WB stage. To maintain single-cycle throughput without stalling, the core implements a comprehensive forwarding network that routes both General Purpose Registers (GPRs) and CSRs across the pipeline stages.

C. FPGA Deployment and Open-Source Toolchain

A significant challenge in hardware security research is the reproducibility of the development environment, which is often plagued by proprietary vendor lock-in, complex dependency resolution, and restrictive licensing. To address this, the entire eSC-V project is packaged using Nix Flakes, providing a fully reproducible, declarative development environment. The complete eSC-V RTL, Nix environment, and test suites are open-source and available on GitHub [19].

The Nix configuration seamlessly isolates and provisions all necessary dependencies, including dual GNU GCC cross-compilers: one for the baseline rv32i-zicsr architecture and a custom toolchain compiled with zicfilp and zicfiss support [7]. This approach allows external researchers and developers to instantly replicate the exact synthesis, simulation, and software compilation environment with a single command, eliminating toolchain friction.

The FPGA development workflow utilizes a suite of open-source EDA tools to transition from VHDL source code to a functional bitstream. The process begins with Simulation and Elaboration, where GHDL handles the VHDL simulation and GTKWave is used to visualize the resulting waveforms. For Synthesis, Yosys performs the RTL synthesis, leveraging the *yosys-ghdl* plugin to parse the VHDL source into an intermediate representation suitable for logic optimization. The workflow then proceeds to Place and Route (PnR), where *nextpnr* manages the physical layout and routing specific to the target FPGA fabric. Finally, Bitstream Generation is completed using Project Apicula (Apycula), an open-source reverse-engineering effort specifically tailored for Gowin FPGAs [9] [10].

D. Hardware Target

The synthesized eSC-V SoC is deployed on the Sipeed Tang Primer 20K development board [8] [11], which features a Gowin GW2A FPGA. The design comfortably meets timing requirements, achieving a stable core clock frequency of 60 MHz. Finally, the generated bitstream is flashed to the FPGA via JTAG using openFPGALoader. By utilizing this end-to-end open-source methodology, eSC-V ensures that the hardware root-of-trust remains completely auditable from the high-level VHDL implementation down to the physical silicon configuration. Verification:

Verification:

Implementing architectural security demands rigorous validation to ensure new vulnerabilities are not inadvertently introduced. Accordingly, eSC-V underwent both formal ISA compliance testing and practical exploit mitigation demonstrations.

The baseline architecture was formally verified using the RISC-V Compatibility Framework (RISCOF) [12], [17], which evaluates a Device Under Test (DUT) against a Golden Reference Model. By executing standardized assembly programs from the riscv-arch-test repository [13] on both models, RISCOF generates and compares deterministic execution signatures (memory dumps) to detect any microarchitectural mismatches or invalid state updates.

Ultimately, the eSC-V processor passed 100% of the applicable compliance tests for the RV32I base instruction set and *Zicsr* extension. This formal verification proves the custom five-stage pipeline correctly handles all baseline RISC-V semantics, establishing a functionally stable foundation for the hardware-enforced CFI primitives.

The Cost of Security:

Table 1. Comparison of Baseline RV32I Core and CFI-enabled RV32I Core

Parameter	Baseline	CFI-Enabled	Growth/Penalty
Logic Utilization (LUTs)	7,007 cells	7,477 cells	+6.71% Area
Max Frequency	61 MHz	58 MHz	-4.92% Speed
Software Code Size	19,072 Bytes	22,853 Bytes	+19.82% Size
Total Registers	2,461 cells	2,469 cells	+0.33% Registers

Integrating hardware-enforced Control-Flow Integrity inherently introduces microarchitectural trade-offs. To quantify this impact, the eSC-V core was synthesized and profiled against a baseline RV32I *Zicsr* configuration lacking the *Zicfilp* and *Smcfiss* extensions (see Table 1).

As the results illustrate, the hardware area penalty is remarkably minimal. The necessary structural additions—primarily the 20-bit label comparator in the execution stage, the two-stage flush tracking logic, and the passthrough PMP unit—consume very little physical silicon, leaving the total register count virtually unchanged.

While there is a slight degradation in maximum clock frequency, this reflects a lengthened critical path caused by the continuous, inline combinatorial access checks required for PMP bounds verification. Despite this minor reduction, the operating frequency remains highly practical and performant for bare-metal embedded applications.

The most pronounced trade-off occurs in the software code size. This footprint expansion is an unavoidable characteristic of standardized CFI: the compiler must instrument the binary

with *lpad* instructions at every valid indirect branch target, alongside *sspush* and *sspopchk* instructions in function prologues and epilogues. Additionally, this overhead incorporates actively developing *newlib* (nano spec) libraries, which still require substantial maturation.

Ultimately, these synthesis results prove that eSC-V successfully trades a moderate firmware footprint increase for near-absolute protection against code-reuse attacks, all while preserving the core logic budget.

Exploit Mitigation:

To validate the real-world efficacy of the eSC-V hardware security extensions, a proof-of-concept (PoC) exploit environment was deployed on the FPGA. A bare-metal C program was developed containing a *vulnerable_function()* that accepts input over the UART interface without proper bounds checking. The objective of the exploit is to execute a target *win_function()* that is never legitimately called during standard program execution.

In the unprotected baseline configuration, a Python script delivers a crafted payload over UART. The payload overflows the local buffer, explicitly overwriting the stored return address or function pointer on the stack with the address of *win_function()*, which is printed out at the start. Upon returning from the vulnerable function, the corrupted address is loaded into the program counter. The exploit succeeds, redirecting control flow and printing the "win" output to the terminal. This confirms that the baseline microarchitecture is fully susceptible to Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP) attacks.

To establish the baseline vulnerability, Figure 2 shows the system terminal log when hardware protection is disabled. The malicious payload successfully overwrites the saved return address on the stack, hijacking the control flow to execute the unintended target and printing the exploit success message.

```

> python3 scripts/exploit.py

[SYSTEM] Smcfiss (Shadow Stack) DISABLED.
[INFO] Address of win_function: 0x8000005C
Enter payload (Overflow past 16 bytes to overwrite saved RA):
>
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[SYSTEM] vulnerable_function() returning...
[!] CRITICAL: ROP EXPLOIT SUCCESSFUL!
[!] Control flow hijacked via Return Address Overwrite!

```

Fig. 2. Exploit success to run *win_function()*

Conversely, Figure 3 presents the system behavior with the eSC-V security extensions enabled. As Figure 3 shows, rather than executing the malicious payload, the hardware enforcement successfully traps the violation. The eSC-V core initiates the two-stage pipeline flush,

neutralizing the faulting instruction, and redirects execution to the *mtvec* trap handler. The system then outputs a fatal crash report via UART, detailing the *mcause* (*0x00000012* for a Zicfilp Software Check Exception), the exact address of the faulting instruction (*mepc*), and a stack trace. This proves that the eSC-V hardware definitively neutralizes code-reuse attacks in M-mode without requiring software-level intervention.

```

> python3 scripts/exploit.py

[SYSTEM] Smcfiss (Shadow Stack) ENABLED.
[INFO] Address of win_function: 0x800000B0
Enter payload (Overflow past 16 bytes to overwrite saved RA):
>
AAAAAAAAAAAAAAAA
[SYSTEM] vulnerable_function() returning...
!!! SYSTEM TRAP (CRASH) !!!
mcause: 0x00000012 (Exception)
- Software check exception
mepc: 0x800001D8 (Address where crash happened)
-----

```

Fig. 3. Successful Trap to mitigate exploit

Toolchain Limitations and Ecosystem Readiness:

Software support for the Zicfilp and Smcfiss extensions remains in its infancy. Although upstream RISC-V GCC branches provide initial CFI encoding support, standard C library integration poses significant challenges.

For eSC-V, extensive custom toolchain compilation was necessary. Furthermore, missing architectural support in standard library initialization prevents using a full *newlib* environment with M-mode CFI. Consequently, firmware validation and exploit proofs relied on minimal bare-metal C and inline assembly.

These hurdles highlight a critical ecosystem gap: while eSC-V proves CFI can be efficiently enforced in hardware, the upstream compiler and standard libraries require substantial maturation before broad adoption in standard embedded workflows.

Comparative Analysis:

A. Microcontroller vs. Application-Class Implementations

While recent implementations like CVA6-CFI [1] target complex, Linux-capable RV64GC systems reliant on virtual memory (MMU/TLB) and privilege transitions, eSC-V aggressively optimizes standardized CFI for bare-metal IoT nodes. Operating exclusively in M-Mode within a

lightweight five-stage pipeline, eSC-V bypasses virtual memory overhead by enforcing shadow stacks directly in the physical address space via Physical Memory Protection (PMP).

B. Standardized ISA vs. Legacy Custom Architectures

Prior research prototypes relied on heavy, non-standard modifications to mitigate code-reuse attacks. In the broader landscape of hardware security research, projects like FIXER [14] and SOFIA [15] illustrate the varied architectural trade-offs required to enforce integrity.

FIXER [14] achieves its goals through the use of dedicated security coprocessors, a design choice that unfortunately necessitates custom software toolchains for implementation. On the other hand, SOFIA [15] adopts a more invasive strategy, demanding full instruction stream encryption and requiring extensive, complex redesigns of the memory subsystem to ensure protection.

In contrast, eSC-V strictly leverages standardized RISC-V ISA extensions. This eliminates proprietary instruction encodings, ensures direct upstream GCC and LLVM toolchain compatibility, and successfully bridges the gap between theoretical hardware security and scalable, real-world deployment.

Conclusion:

The eSC-V is a microcontroller-class RV32I processor developed in VHDL to evaluate hardware-enforced Control-Flow Integrity (CFI) within bare-metal environments. By integrating standardized extensions like the ratified Zicfilp for forward-edge protection and the draft Smcfiss and Smpmpind for backward-edge security, the architecture demonstrates that robust protection is achievable without operating system support. The design employs a classic five-stage in-order pipeline enhanced with a custom two-stage flush mechanism to handle execution faults safely and a passthrough Physical Memory Protection (PMP) unit to isolate the Machine-mode shadow stack. Verified against formal models like Sail and Spike using the RISCOF framework, eSC-V proves that standardized CFI can be efficiently implemented in resource-constrained systems with negligible logic overhead.

Successfully deployed on a Tang Primer 20K FPGA at 60 MHz, the project offers a fully reproducible, open-source hardware reference packaged via Nix flakes to eliminate vendor lock-in. While the implementation confirms the viability of securing next-generation IoT and robotics platforms against code-reuse attacks, it also identifies a significant gap in ecosystem readiness due to the extensive custom compilation required for these extensions. By documenting these requirements and providing a transparent, auditable reference architecture, this work underscores the necessity for upstream compiler and standard library maturation. Ultimately, eSC-V serves as a practical blueprint for integrating advanced security features into safety-critical, bare-metal RISC-V platforms using an entirely open-source EDA flow.

References:

1. Manoni S., et al.: [CVA6-CFI: A first glance at RISC-V control-flow integrity extensions](#). arXiv preprint (2026). [arXiv:2602.04991](#) **(Article within Journal)**
2. Maunero N., Prinetto P., Roascio G., et al.: [A FPGA-based Control-Flow Integrity Solution for Securing Bare-Metal Embedded Systems](#). 2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS), 1-10 (2020). [10.1109/DTIS48698.2020.9081314](#) **(Conference Paper)**
3. Mishra T., Chantem T., and Gerdes R.: [Survey of Control-flow Integrity Techniques for Real-time Embedded Systems](#). ACM Transactions on Embedded Computing Systems 21, 1–32 (2022). [10.1145/3538275](#) **(Article within Journal)**
4. Elshiemy M.A., and Soltan A.: [Comprehensive Compliance Framework for RISC-V Architectures using the RISCOF Test Platform](#). 2025 IEEE International Symposium on Circuits and Systems (ISCAS), 1-5 (2025). [10.1109/ISCAS56072.2025.11043314](#) **(Conference Paper)**
5. De A., Basu A., Ghosh S., and Jaeger T.: [FIXER: Flow Integrity Extensions for Embedded RISC-V](#). 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), [348-353 \(2019\)](#). [10.23919/DATE.2019.8714980](#) **(Conference Paper)**
6. de Clercq R., et al.: SOFIA: [Software and control flow integrity architecture](#). 2016 Design, Automation & Test in Europe Conference & Exhibition (DATE), (2016). **(Conference Paper)**
7. Walls R.J., Brown N.F., Le Baron T., et al.: [Control-flow integrity for real-time embedded systems](#). 31st Euromicro Conf. on Real-Time Systems (ECRTS) (2019). **(Conference Paper)**
8. RISC-V International. (2026). Ratified extensions. RISC-V Tech Hub. URL: <https://riscv.atlassian.net/wiki/spaces/HOME/pages/16154732/Ratified+Extensions> **(Website)**
9. RISC-V International. (2026). Control-flow integrity (CFI). RISC-V Ratified Specifications Library. URL: <https://docs.riscv.org/reference/isa/priv/priv-cfi.html> **(Website)**
10. RISC-V International. (2024, June). RISC-V ratifies additional specifications in June board meeting. URL: <https://lists.riscv.org/g/tech-announce/message/474> (Website)
11. GCC GNU Compiler Collection. (2024, November). [PATCH 2/4] RISC-V: Add Zicfilp ISA extension. URL: <https://gcc.gnu.org/pipermail/gcc-patches/2024-November/668919.html> **(Website)**
12. Sipeed Wiki. (2026). Tang Primer 20K. URL: <https://wiki.sipeed.com/hardware/en/tang/tang-primer-20k/primer-20k.html> (Website)
13. YosysHQ. (2026). Project Apicula: Bitstream documentation for Gowin FPGAs. GitHub. URL: <https://github.com/YosysHQ/apicula> **(Website)**
14. PyPI. (2026). Apycula, PyPI package documentation. URL: <https://pypi.org/project/Apycula/0.10.0/> **(Website)**

15. Hackaday. (2022, June 12). An open toolchain for Sipeed Tang Nano FPGAs. URL: <https://hackaday.com/2022/06/12/an-open-toolchain-for-sipeed-tang-nano-fpgas/> (**Website**)
16. RISCOF. (2026). RISCOF 1.23.4 Documentation. URL: <https://riscof.readthedocs.io/en/latest/intro.html> (**Website**)
17. RISC-V Non-ISA. (2026). riscv-arch-test repository. GitHub. URL: <https://github.com/riscv-non-isa/riscv-arch-test> (**Website**)
18. Ved-rivos. (2026). RISC-V ISA-MANUAL Extension for Machine-mode Shadow Stack. GitHub. URL: <https://github.com/ved-rivos/riscv-isa-manual> (**Website**)
19. ethycS0. (2024). eSC-V: RV32I SoC in VHDL with Hardware-enforced CFI. GitHub. URL: <https://github.com/ethycS0/eSC-V> (**Website**)